

An Empirical Evaluation of Message Delivery Reliability and Recovery Characteristics in Redis Streams and NATS JetStream

Victor Bona

Independent Researcher, Blumenau, Brazil

`victor.bona@hotmail.com`

September 2026

Abstract

Message retention does not by itself determine how quickly work recovers after a consumer fails, nor what a publication confirmation guarantees. We present an auditable, computationally reproducible case study of recovery policies, concurrency behavior, and acknowledgment costs in Redis Streams 7.4.2 and NATS JetStream 2.10.24. Measurements use single-replica deployments and 512-byte messages on a two-node Kubernetes homelab. The study comprises 300 randomized backlog-drain trials, 60 planned publication trials, 60 serial-publisher sensitivity trials, 120 consumer-process kills, and ten no-reclaimer controls. Conditional proofs distinguish residual acknowledgment timeout, recovery scheduling, consumer occupancy, stable-storage assumptions, and the external-effect/acknowledgment gap; selected model statements are checked with Lean 4.24.0. All active recovery trials completed. At a 250 ms timeout, kills at 10% and 75% of the receipt-based interval were followed by median failure-to-redelivery intervals of 226.5 and 63.6 ms for JetStream and 230.2 and 67.9 ms for Redis with a 10 ms reclaim sleep. Redis controls without reclamation remained pending. With sixteen consumers, mean acknowledged drain throughput was 8.59–12.94 times the one-consumer mean across profiles, alongside higher mean within-trial cycle p99 latency. In the sixteen-publisher workload, periodic configurations had approximately 83% and 82% of memory-mode throughput for Redis and JetStream, respectively. Always-synchronize profiles exhibited lower throughput and greater variability; one Redis-always timed trial failed and is retained as an outcome. Completed-run estimates condition on completion. Sensitivity diagnostics retain the direction of paired contrasts but expose influential observations; storage-history carryover prevents isolating synchronization-policy effects. Consumer-state persistence differs between products, and consumer kills do not validate power-loss durability. The versioned artifact retains raw timings, failures, configurations, proofs, and deterministic analysis for this bounded comparison.

Keywords: message delivery; Redis Streams; NATS JetStream; consumer failure; redelivery latency; persistence; reproducible benchmarking.

1 Introduction

A consumer can disappear after receiving a message but before acknowledging it. Recovering that message requires more than retaining its payload: the system must preserve the pending-delivery state, make the message eligible again, and supply it to an available consumer. The resulting delay depends on both broker policy and client behavior. Conversely, a fast acknowledgment

can indicate acceptance without establishing that either a message or a consumer’s progress has reached stable storage. These distinctions make a single throughput number an incomplete basis for selecting a messaging system.

Redis Streams and NATS JetStream offer persistent streams and explicitly acknowledged consumption, but expose different recovery mechanisms. In the Redis release evaluated here, a consumer group records outstanding work in a pending-entry list, and clients can transfer eligible entries through `XAUTOCLAIM`. JetStream tracks acknowledgment deadlines in a durable consumer and can redeliver expired work when pull demand exists [18, 29]. Neither mechanism is an atomic transaction with an application’s external effects. This is an instance of a longstanding end-to-end problem: a transport’s delivery acknowledgment cannot alone establish completion of an application operation [21, 1].

This study asks three questions:

RQ1: How does consumer failure affect message redelivery latency?

RQ2: How do both systems behave under increasing consumer concurrency?

RQ3: What are the throughput/latency trade-offs under different durability configurations?

We answer them with a descriptive, single-replica homelab study of Redis 7.4.2 and NATS Server 2.10.24. The contribution is an auditable, computationally reproducible comparison of specific recovery policies, concurrency behavior, and acknowledgment costs under documented conditions. We combine source-level contract analysis, conditional mathematical proofs, process-kill experiments, randomized repeated performance trials, and retained per-message timing records. Three separate workloads evaluate recovery, backlog consumption, and confirmed publication. The artifact connects each reported estimate to its observations and each formal claim to its premises. The resulting comparison concerns these client paths, workloads, and deployment; it establishes neither a universal broker ranking nor equivalent durability guarantees.

The main concurrency experiment contains 300 trials and 4,915,200 measured messages. The separate durability experiment has 60 planned trials, with failures retained as outcomes; the serial-publisher sensitivity has 60 trials. Recovery uses 120 consumer-kill trials and ten negative controls. Each experiment is executed in a dedicated namespace on the same authorized Kubernetes homelab; the workstation is used for authoring, analysis, and document compilation. No external broker service or production messaging instance participates.

The distinction between proof and observation is essential. A mathematical theorem can establish a conditional timing relation or expose a delivery/acknowledgment failure window. A finite benchmark can estimate performance under its measured conditions; it cannot prove an implementation correct for every execution or establish a hardware-independent performance ordering. We give proofs where those claims are provable, and report repeated measurements, uncertainty, and limitations for the empirical claims. Section 9 answers each research question at that level of evidence.

2 Background and related work

Delivery guarantees combine distinct safety and liveness claims. Retaining an unacknowledged message is a safety property; eventually assigning it to a live consumer additionally requires a recovery policy and progress assumptions. At-least-once delivery permits duplicates and does not establish exactly-once application effects. We distinguish publication confirmation, client receipt,

broker processing of a consumer acknowledgment, and completion of an external effect [1, 21]. Persistence describes which state is intended to survive a stated failure class. A consumer-process failure, broker-process failure, pod deletion, and host power loss therefore cannot be treated as interchangeable tests of “durability.”

2.1 Redis Streams

`XADD` appends an entry to a stream. With acknowledgment tracking enabled, `XREADGROUP` assigns an entry to a consumer and records it in the group’s pending-entry list (PEL). `XACK` removes the corresponding pending record; it does not delete the stream entry [14, 12]. A new-message read with the identifier `>` does not itself transfer another consumer’s pending entries. Consequently, consumer death and message reassignment are separate events.

In Redis 7.4.2, `XAUTOCLAIM` scans pending entries and transfers those whose idle age reaches the configured minimum. It exposes a continuation cursor; a general reclaimer must advance that cursor and revisit the list. A call scans at most ten times its `COUNT` bound, so an arbitrary-size PEL cannot be treated as a single timer event [13, 18]. Our recovery experiment deliberately has one pending message and `COUNT=1`. This isolates timeout and polling behavior, but does not evaluate a large pending-list scan. Newer Redis mechanisms such as `XREADGROUP CLAIM` are outside the selected release and are not used.

Redis persistence is configured separately from stream consumption. We disable snapshots and compare no append-only file (AOF), AOF with `appendfsync everysec`, and AOF with `appendfsync always`. The last setting can group commands into a shared synchronization operation; it is not necessarily one physical synchronization per message. The nominal every-second mode does not prove a hard one-second power-loss window: the pinned implementation can postpone writes while a previous background synchronization is in progress, and I/O or scheduling delay remains possible [11, 16, 15].

2.2 NATS JetStream

JetStream stores messages in streams and records delivery progress in consumers. We use one durable pull consumer shared by all workers, explicit per-message acknowledgment, no acknowledgment backoff, and one stream replica. Workers bind to this same consumer instead of creating independent durable consumers, which would represent separate subscriptions rather than competing workers [25].

`AckWait` measures time from delivery, not from notification of consumer death. In the pinned server, an expired pending entry is queued for redelivery; an available pull request is still necessary to obtain it. The timeout helper introduces a nominal 1 ms timer delay, and scheduling can add more [29]. Our Go client uses `AckSync`, which waits for a server response to the acknowledgment. This establishes acknowledgment processing, not an atomic business transaction and not, by itself, a synchronous stable-storage barrier for consumer state [27].

This last qualification matters for the durability comparison. The selected server accepts `sync_interval: always` for file storage, whereas its default interval is two minutes; our periodic configuration explicitly changes that interval to one second [28, 30]. Stream append synchronization and consumer-state persistence follow different paths. For a single-replica consumer, acknowledgment updates can be handed to an asynchronous state flusher before the acknowledgment response; even `always` mode does not turn every such response into a synchronous consumer-state write [29, 30]. Accordingly, the two systems’ “always” profiles belong to a com-

parable append-policy class, but do not provide interchangeable consumer-progress persistence contracts.

2.3 Prior empirical and methodological work

Lim, Gu, and Yoon compare RabbitMQ, Kafka, Redis Streams, and NATS JetStream in a Kubernetes autoscaling study [8]. The study evaluates KEDA-driven consumer autoscaling, scaling response, resource use, throughput, and latency. Our study instead holds consumer concurrency fixed within each trial and separately varies consumer-failure timing and persistence policy. The reported experiments in that study do not include the consumer-kill or persistence-policy interventions examined here. The workloads and scaling policies differ, so the present measurements are neither a replication nor a contradiction of its results. We make no first-comparison claim. Paul et al.’s broker benchmark is also relevant, but its Redis and NATS paths use Pub/Sub and Core NATS; those measurements do not establish Streams or JetStream recovery semantics [10]. Coenen et al.’s financial-feed benchmark motivates reproducible workload definition, while addressing a different application setting [3].

Jepsen’s NATS 2.12.1 report investigates stronger faults, including simulated power failures and replicated deployments [7]. It is an original technical report, not a peer-reviewed study, and concerns a different release and configuration. Its fault distinctions reinforce why killing a consumer while the broker remains alive is insufficient evidence for power-loss durability. We do not transfer its outcomes to our deployment.

Our repetition and uncertainty analysis follows the principle that benchmark variation has several levels and that many observations within one execution do not substitute for repeated executions [6]. Randomized block order reduces the association between treatment and time, although it cannot eliminate every environmental bias [9]. We explicitly distinguish finite backlogs and closed-loop publication from open-loop offered loads, following the warning that these workload models can yield different conclusions [22]. Our latencies are not coordinated-omission-corrected production SLA estimates [23, 5].

3 System model and formal results

3.1 Events, failure scope, and assumptions

An application assigns each message a unique identifier. We distinguish publication invocation p , publication confirmation q , broker delivery d , client receipt $\hat{d}$, consumer failure f , redelivery receipt $\hat{r}$, and acknowledgment completion a . All measured events use a common Linux monotonic clock on the driver node. The broker’s internal delivery timestamp d is not directly observed; replacing it with $\hat{d}$ without qualification would misstate the timer origin.

The fault model is a fail-stop consumer process killed with **SIGKILL** while its message is unacknowledged. The broker, network, retained payload, group/consumer state, and surviving consumer remain available. There is no message expiry, trimming, competing claim, negative acknowledgment, acknowledgment extension, or backoff change during the recovery episode. The timeout threshold is $\Delta > 0$ and failure occurs at age $h = f - d$, with $0 \leq h < \Delta$. These are assumptions of the isolated formal model; they do not characterize every production execution. The controller sets age relative to client receipt, so exact broker age is unobserved. Idealized eligibility also abstracts from timer quantization and assumes no relevant broker-clock discontinuity.

For a bounded redelivery theorem, additional timing assumptions are required. Let P bound the time from eligibility until a reclaimer or expiry mechanism selects the entry, and let B bound subsequent scheduling, demand availability, transport, and client observation. Redis command execution time and scheduler delay must be included in P ; its configured sleep alone is not such a bound. JetStream also requires pull demand, so B cannot silently be set to zero.

Theorem 3.1 (Residual timeout and conditional recovery bound). *Under the preceding assumptions, let selection be forbidden before $d + \Delta$, occur at some $s \in [d + \Delta, d + \Delta + P]$, and be observed after an additional delay $b \in [0, B]$. Then*

$$\Delta - h \leq \hat{r} - f \leq \Delta - h + P + B.$$

If the corresponding upper-bound premise is removed and admissible selection or observation delays are unbounded, no finite uniform upper bound on redelivery latency follows.

Proof. By definition, $\hat{r} = s + b$ and $f = d + h$. Hence

$$\hat{r} - f = \Delta - h + (s - (d + \Delta)) + b.$$

The two final terms lie in $[0, P]$ and $[0, B]$, respectively, proving both inequalities. If either delay is unbounded, for any proposed finite upper bound choose an admissible delay exceeding it. The corresponding execution violates that proposed bound without violating eligibility or retention. $\square$

Corollary 3.2 (Why client-observed timer excess can be negative). *Write $\hat{d} = d + u$, where $u \geq 0$ is the initial delivery-to-receipt delay, and let $w = s - (d + \Delta)$. Then*

$$(\hat{r} - \hat{d}) - \Delta = w + b - u.$$

Thus a measured receipt-to-redelivery interval slightly below Δ does not alone refute server-side timeout eligibility.

Proof. Substitute $\hat{r} = d + \Delta + w + b$ and $\hat{d} = d + u$, and cancel d . $\square$

Proposition 3.3 (No autonomous Redis reassignment from new-message reads). *In the stated Redis 7.4.2 model, there exists an infinite execution with an available broker and surviving consumer, a retained message pending for a failed consumer, and no redelivery, if the survivor performs only new-message reads and no reclamation or pending-history recovery.*

Proof. Deliver the only message to the consumer that subsequently fails, thereby placing it in that consumer's PEL history. Repeat `XREADGROUP` with `>` from the survivor. Each read considers previously undelivered entries, while the retained entry remains pending for the failed consumer. With no further publication or state-changing recovery operation, every repetition preserves this state. The infinite repetition witnesses the claim. $\square$

The proposition is a liveness counterexample, not a defect claim. A Redis application supplies the missing recovery operation. Likewise, a JetStream pull consumer without future pull demand has no unconditional delivery-time guarantee. The experimental contrast concerns who initiates eligibility/reassignment and the timing of the configured client policy.

3.2 Concurrency and finite-window throughput

Definition 3.4. For $N > 0$ completed messages, an integer $C \geq 1$ of consumers, and a drain interval of duration $T > 0$, let $X = N/T$ be acknowledged throughput. Message i occupies one consumer during a nonoverlapping-on-that-consumer interval $[r_i, a_i]$, from its successful receive-request invocation to acknowledgment completion. Write $R_i = a_i - r_i$ and $\bar{R} = N^{-1} \sum_i R_i$.

Theorem 3.5 (Consumer occupancy bound). *If every interval lies within the measured drain window and each of C consumers has at most one such interval active at a time, then*

$$X\bar{R} \leq C.$$

More precisely, with $U = (CT)^{-1} \sum_i R_i \in [0, 1]$, one has $X = CU/\bar{R}$ whenever $N > 0$ and $\bar{R} > 0$.

Proof. For consumer j , its intervals have disjoint interiors and are contained in a window of length T , so their lengths sum to at most T . Summing over all C consumers gives $\sum_i R_i \leq CT$. Since $X\bar{R} = (N/T)(\sum_i R_i/N) = \sum_i R_i/T$, the inequality follows. Substituting the definition of U yields the identity. $\square$

For two individual trials with positive mean cycle times, this gives the exact explanatory decomposition:

$$\frac{X(C_2)}{X(C_1)} = \frac{C_2}{C_1} \frac{U(C_2)}{U(C_1)} \frac{\bar{R}(C_1)}{\bar{R}(C_2)}.$$

The identity does not generally survive separately averaging throughput, occupancy, and cycle time across trials. Reported ratios of condition-mean throughputs are distinct summaries. Approximately stable cycle cost and occupancy are sufficient for approximately linear scaling within this model. More generally, linear scaling requires $U(C)/\bar{R}(C)$ to remain approximately constant; increasing C alone does not prove it. For example, admissible conditions with $U = 1$, $C_1 = 1$, $\bar{R}(C_1) = 1$, $C_2 = 2$, and $\bar{R}(C_2) = 3$ give throughputs 1 and $2/3$. This counterexample rules out a universal monotonicity claim under unrestricted contention. Our analysis checks the occupancy inequality directly against every retained concurrency-trial event interval.

3.3 Acknowledgment, storage, and application effects

Proposition 3.6 (Conditional stable-storage implication). *Assume (i) a successful synchronization makes the message and all metadata required for its recovery stable, (ii) subsequent failures preserve that stable state, (iii) recovery interprets it correctly, and (iv) confirmation occurs only after that synchronization completes successfully. Then every confirmed message is recoverable after such a failure. If confirmation is instead permitted before stabilization, confirmation alone does not imply recoverability.*

Proof. Under (iv), every confirmed message has already undergone the successful synchronization of (i). Its recovery state survives by (ii), and (iii) recovers it. For the second claim, choose an allowed execution in which confirmation precedes synchronization and a failure occurs strictly between them. Let the failure discard all unstable state. The message was confirmed but is not recoverable, providing a counterexample. $\square$

This is a theorem about a storage contract, not a formal verification of either product. In particular, the pinned NATS append path invokes synchronization without checking its returned

error in that path, whereas the corresponding Redis always-mode path treats a synchronization error as fatal [30, 16]. The experiment injects no I/O faults and does not establish assumptions (i)–(iv) for every implementation execution. The source distinction prevents us from treating a mode name as an unconditional proof of identical fault handling.

Consider an interval containing $F \geq 1$ complete, nonoverlapping synchronization operations. If each occupies at least $f_{\min} > 0$ time, every counted confirmation is assigned to one completed operation, and at most an integer $b \geq 1$ of confirmations are assigned to each operation, those operations can confirm at most bF messages while occupying at least $Ff_{\min}$ time. Their service rate is therefore at most $b/f_{\min}$. This bound applies to the explicitly serialized synchronization stage; overlapping stages and the rest of the pipeline require a different model. It explains why group commit and outstanding-publication count can affect cost without asserting a fixed number of hardware synchronization calls per message.

Proposition 3.7 (The application-effect/acknowledgment gap). *For a non-idempotent external effect and a broker acknowledgment that are separate operations, changing only their order cannot guarantee exactly one completed effect under arbitrary fail-stop interruption and eventual redelivery of unacknowledged work.*

Proof. If the effect precedes acknowledgment, interrupt after the effect and before acknowledgment. The retained unacknowledged message may be redelivered and its effect repeated. If acknowledgment precedes the effect, interrupt after acknowledgment and before the effect; the broker has no remaining obligation to redeliver, so the effect can be absent. Either order admits a counterexample. An atomic transaction, durable application deduplication, or an idempotent effect adds an assumption not present in this model. $\square$

3.4 What zero observed failures establishes

For $n \geq 1$ independent identically distributed Bernoulli trials with failure probability p , observing zero failures has probability $(1 - p)^n$. For $0 < \alpha < 1$, setting this to α gives the one-sided exact upper confidence limit

$$p_{\text{upper}} = 1 - \alpha^{1/n}.$$

At $n = 10$ and $\alpha = 0.05$, this is approximately 0.259, not zero [2]. Our heterogeneous configurations and temporally adjacent messages are not automatically independent Bernoulli draws from one population. We therefore report exact observed counts and do not turn millions of message completions into a misleading claim of extremely small universal failure probability.

3.5 Machine-checked subset and proof boundary

The artifact additionally formalizes selected model statements in Lean 4.24.0 with its bundled `Std` library [4, 24]. The integer-time residual bound and receipt-offset identities, the division-free sum of per-consumer busy times, and explicit finite effect/acknowledgment crash counterexamples form ten checked declarations. Verification ran on the homelab after all benchmark collection had ended, with warnings treated as errors; the final source digest, compiler version, exit status, and transitive logical dependencies are retained. An initial unused-binder warning was resolved by renaming that binder without changing any hypothesis. A fresh compilation of the unchanged source in a separate homelab namespace on 24 September 2026 again passed all ten declarations with the same transitive logical dependencies; both verification receipts are retained.

The proof dependencies are the standard Lean principles `propext`, `Quot.sound`, and, in one offset equivalence, `Classical.choice`; the four finite-trace declarations use no axioms. No admitted proof, custom axiom, or native-evaluation shortcut is used. This formalization does not verify either broker implementation, the measurement software, real-number division, statistical inference, storage survival, or the infinite new-read-only execution. Those distinctions remain governed by the explicit paper proofs, source evidence, and empirical checks. A successful Lean check therefore strengthens the selected abstract arguments without upgrading an empirical observation into a universal system guarantee.

4 Experimental method

4.1 Testbed, versions, and resource isolation

All experiment workloads run in the `homelab` Kubernetes context. The dedicated namespace is `msgrel-20260923`. The broker pods are fixed to `homelab-01`; the driver is fixed to `homelab-02`. The broker node exposes 16 logical CPUs from an AMD Ryzen 7 5825U and approximately 32 GiB of RAM. Both nodes run NixOS 24.05, Linux 6.6.68, K3s v1.30.4+k3s1, and containerd 1.7.20-k3s1. The driver node exposes four logical CPUs from an Intel N100 and approximately 16 GiB of RAM. Exact capacities, runtime details, and baseline utilization are retained in the environment receipts.

Each broker has a 2-CPU and 1-GiB memory limit. The driver has a 2-CPU and 2-GiB limit, and its Go runtime uses `GOMAXPROCS=2`. It communicates with the brokers through cluster-internal services across the two nodes. No TLS or application authentication is included in the measured path. A namespace ingress policy permits only experiment-pod ingress; no broker is exposed externally. Pods have no mounted service-account credentials. The homelab hosts other services, which remain untouched. It is a shared testbed, not exclusive hardware or a CPU-pinned laboratory.

The disk profiles use disk-backed Kubernetes `emptyDir` volumes on the same broker node. The observed backing filesystem is Btrfs on an NVMe device, mounted with `compress=zstd:3`. This is a filesystem compression setting, not a measured per-file compression ratio. This storage survives a container restart but is deleted with its pod; it is not a persistent-volume or host-failure experiment. We neither infer the drive’s power-loss protection from its name nor claim to have validated its hardware cache semantics. The memory profile has no configured persistent stream payload.

Redis is pinned to 7.4.2 and NATS Server to 2.10.24; these are deliberately historical releases supporting all evaluated mechanisms, not claims about the latest releases [17, 31]. The driver is compiled inside the homelab using Go 1.24.4, `go-redis/v9` 9.7.0, `nats.go` 1.39.1, and `x/sys` 0.29.0. Exact image digests and transitive Go module checksums accompany the artifact. These details constrain reproducibility more precisely than product names alone.

For Redis, snapshots and automatic AOF rewrite are disabled. The option `no-appendfsync-on-rewrite` remains `no`; the memory policy is `noeviction`. These choices exclude rewrite interference and eviction from this bounded experiment. JetStream uses limits retention with no message-count, byte, or age expiration within the test stream. An acknowledgment removes pending state but retains the source message in both systems. There is no replication comparison.

Logical stream cleanup does not reset persistent storage history. Redis appends stream creation, message, acknowledgment, and deletion commands to the same AOF throughout the campaign, with automatic rewriting disabled. JetStream deletes the separate stream store

Profile	Redis 7.4.2	JetStream 2.10.24
Memory (M)	Snapshots off; AOF off	Memory stream; one replica
Periodic (P)	AOF on; <code>appendfsync everysec</code>	File; <code>sync_interval: 1s</code>
Always (S)	AOF on; <code>appendfsync always</code>	File; <code>sync_interval: always</code>

Table 1: Configured append-persistence classes. “Always” requests synchronous append behavior; the consumer-state and I/O-error qualifications in Sections 2 and 3 remain applicable. All profiles use one broker replica.

after each trial. These treatments therefore accumulate different storage histories even though their live logical streams are removed. The final Redis-periodic and Redis-always receipts report AOF sizes of 2,590,379,564 and 907,510,749 bytes, respectively. These are reported AOF lengths, not measured physical allocation after filesystem compression. Redis-periodic also reports `aof_delayed_fsync=1`, whereas Redis-always reports zero. These are cumulative final-state diagnostics, not time-resolved evidence assigning a particular stall to AOF growth or delayed synchronization. Randomized order does not reset this accumulated state; performance estimates include the interaction between configured policy and the evolving storage environment.

4.2 Workload and timing definitions

Each message contains exactly 512 application payload bytes, including an eight-byte identifier unique within its trial; all remaining bytes are the literal character `x`. The payload is therefore highly compressible, which matters for the configured filesystem. Protocol framing and metadata are additional bytes and differ between products. Publication uses confirmed `XADD` calls or JetStream `Publish` calls awaiting `PubAck`. Each publisher has one request outstanding and its own TCP connection. Consumption uses `XREADGROUP COUNT 1` followed by `XACK`, or `Fetch(1)` followed by `AckSync`, with one outstanding receive/acknowledgment cycle per consumer connection. No artificial application-processing delay is added.

Client timeouts are distinct from broker eligibility timers. Redis uses five-second default socket read and write timeouts for publication and acknowledgment, and 30-second defaults on administrative connections. These are per socket stage, not an absolute whole-call deadline. For a positive blocking `XREADGROUP`, go-redis 9.7.0 instead assigns a socket read timeout of `BLOCK` plus ten seconds [20, 19]. Ordinary drain and control reads use a 100 ms server block and a 10.1 s client socket read timeout; the victim’s initial read uses a 5 s server block and a 15 s socket read timeout. An empty server read ordinarily returns near the shorter blocking interval. NATS publication and acknowledgment use five-second response waits; its `Fetch` calls receive the explicit 100 ms or 5 s client wait. None of these settings is the recovery eligibility timeout Δ .

The primary timestamps and estimands are:

$$\begin{aligned}
L_{\text{pub},i} &= q_i - p_i, & L_{\text{cycle},i} &= a_i - r_i, \\
L_{\text{ack},i} &= a_i - \hat{d}_i, & L_{\text{drain},i} &= \hat{d}_i - t_0, \\
L_{\text{recovery}} &= \hat{r} - f, & X_{\text{drain}} &= \frac{N}{\max_i a_i - t_0}.
\end{aligned}$$

Here t_0 is the common drain start, after all messages have been published and all worker connections prepared. Trailing worker fetch timeouts after the last acknowledgment are excluded from the throughput denominator. For $N \geq 1$ and $0 < p \leq 1$, a nearest-rank percentile selects

sorted observation $\lceil pN \rceil$ at percentile fraction p . A reported mean p99 is the mean of completed trials’ within-trial p99 values, not the p99 of pooled messages.

For each publication trial, we also report its message count N , the selected p99 rank $k = \lceil 0.99N \rceil$, and the number $N - k$ of higher order-statistic positions. The latter counts positions rather than strictly larger values, since observations can tie. A fixed admission duration produces different N across conditions, so the empirical percentiles have different finite-sample resolutions. These counts characterize the information underlying each reported tail statistic; they do not supply an independent-sampling assumption or a population-percentile confidence guarantee.

All benchmark intervals use `CLOCK_MONOTONIC` on the driver node, including timestamps from its consumer child processes. This avoids subtracting wall clocks across hosts. The workload generator’s connection setup, stream/group creation, warm-up, compression, and result serialization are outside the timed intervals. Source streams are distinct between trials and removed afterward. A 256-message excluded warm-up precedes each concurrency, durability, and serial-publisher trial, using a separate stream; it warms relevant execution paths but does not eliminate cold allocations in the measured stream. The single-message recovery trials use their own setup without that warm-up.

4.3 RQ1: consumer failure and redelivery

The recovery series measures redelivery after consumer failure using the periodic profile of each system while the broker remains alive. The driver creates a stream and one shared group/durable consumer, publishes one message, and starts a separate victim process. After receiving the message, that process reports its application ID and receipt timestamp through a pipe and remains alive without acknowledging. The controller then kills it at approximately $\alpha\Delta$ after receipt, with $\Delta \in \{250, 1000\}$ ms and $\alpha \in \{0.10, 0.75\}$. The exact kill-call start, process-exit observation, and termination status are recorded. The start of the kill call is the operational failure-time estimate; its bracket is reported rather than hidden. There is no matched control retaining a live, unacknowledging victim through timeout expiry. The experiment therefore measures recovery after a kill without identifying a separate causal effect of death on redelivery eligibility.

For Redis, a separate live connection issues `XAUTOCLAIM` with `COUNT=1`, cursor 0-0, and minimum idle age Δ , sleeping 10 or 100 ms after unsuccessful replies. Its initial sleep phase is randomized uniformly over the configured sleep interval. Because only one entry is pending, restarting from 0-0 scans the complete relevant PEL; this would not be a general scalable reclamation policy for a large PEL.

For JetStream, the survivor binds to the same durable consumer with explicit acknowledgments, `AckWait= $\Delta$` , no backoff, and a shared `MaxAckPending=32768`. It repeatedly requests one message with a 100 ms client timeout. The pinned Go client does not use an initial `no_wait` probe for `Fetch(1)`, but its request expiration is approximately 90% of that timeout. A pull may consequently expire about 10 ms before the client refreshes it [27]. Recovery therefore includes this client’s possible demand gap; it is not an isolated measurement of the server timer.

Survivor polling starts before the kill, after the victim reports receipt. There are ten repetitions of each of the twelve configurations, in randomized order. A successful recovery requires the same application ID, successful acknowledgment, retained-message count one, and pending count zero. JetStream’s delivery metadata must show exactly two deliveries. The nominal censoring budget is $5\Delta + 2$ s, starting after the controller observes the victim’s exit. The controller then stops the survivor loop and reconciles state after its current operation returns; this can extend the

final observation interval. A trial reaching that budget without recovery is retained as censored, not removed from analysis. Ten Redis controls (five at each timeout, with $\alpha = 0.10$) run only new-message reads, with no reclaimer; their corresponding nominal budget is 2Δ , and final reconciliation verifies that the entry is still pending.

The historical recovery records do not timestamp the timer notification, survivor-loop completion, or final reconciliation. Their exact observation endpoints cannot therefore be reconstructed. We report the controls as non-recovery with pending state retained through that procedure, without assigning exact censor durations or fitting a survival model.

4.4 RQ2: increasing consumer concurrency

The concurrency series contains all six profiles and $C \in \{1, 2, 4, 8, 16\}$, with ten randomized complete blocks. Each measured trial first publishes 16384 messages using 16 publishers, then drains that retained backlog through C independent worker connections. All workers share a single group/durable consumer. Outside the recovery experiment, JetStream uses a 30 s acknowledgment deadline, `MaxAckPending`=32768, and `MaxWaiting`=512; Redis performs no reclamation during these failure-free drains. This establishes a fixed amount of work while removing concurrent producer throughput as a limiting input to the drain phase. It measures the combined client, protocol, transport, broker, and acknowledgment path, not an intrinsic maximum server capacity.

For every drained application ID, the harness records publication, its successful receive-request invocation, receipt, acknowledgment, and worker assignment. Empty or timed-out receive requests are not retained as per-message records. The harness requires the exact set of application IDs, no duplicates, no outstanding acknowledgments, and the expected retained stream length. Analysis reconstructs per-worker intervals and checks Theorem 3.5. Application effects, large batches, application-level compression, custom partitioning, multiple streams, and consumer counts beyond 16 are not measured. In particular, these are concurrent logical consumers implemented by Go goroutines, not a cluster of separately scheduled consumer pods.

4.5 RQ3: separate publication-cost measurement

The primary durability series contains ten randomized complete blocks of the six profiles. Sixteen publisher connections continuously perform confirmed appends during a five-second admission window. Admitted requests are allowed to finish; throughput divides the completed count by the interval from the common start until the last confirmation. Thus trailing completions are included rather than assigned to an artificially short denominator. This interval spans several nominal one-second synchronization cycles, but remains a bounded append workload, not a long soak or proof of stationary performance.

This series verifies the number of issued and confirmed application IDs against retained stream length. It does not read back every stored payload, and its integrity claim is correspondingly narrower than that of the drain series. A separate sensitivity series uses one publisher, 1024 messages, one consumer, and ten blocks. It provides a separate one-publisher workload sensitivity. Differences cannot be attributed solely to publisher count because duration and total workload also differ from the five-second experiment. We make no claim that these profiles have equal recovery-point objectives under a power cut.

4.6 Randomization, uncertainty, and evidence checks

The randomization seed is 20260923. The experimental unit is a trial, with pairing by block for treatment ratios. For each condition we report the arithmetic mean of its completed planned trials and a percentile bootstrap 95% confidence interval using 10000 resamples of those trial indices. Conditions have ten completed trials except Redis-always publication, which has nine completions and one failed planned trial. That failure is reported separately, not assigned an artificial throughput or latency. A paired ratio is the ratio of resampled condition means under the same resampled block indices, not the mean of arbitrary per-message ratios. Comparisons involving a failed block use only the blocks completed by both conditions, so these are conditional-on-completion estimates. The bootstrap seed is fixed. These nominal, conditional confidence intervals summarize uncertainty in the condition mean under the working resampling assumption that completed block vectors are independent, identically distributed replicates. All blocks belong to one deployment campaign; their temporal dependence and the evolving storage history are not estimated away by resampling. They are not prediction intervals for individual runs, do not have proven exact finite-sample coverage here, and do not account for every systematic error or hardware population.

These are descriptive effect estimates, not a collection of significance tests with familywise error claims. We specify no p -value threshold, do not stop when an interval becomes favorable, and do not pool individual messages as independent replications. Recovery tables report all ten values through medians and ranges rather than presenting a high percentile from ten failure episodes as a precise tail estimate. No finite trial count can prove absence of all possible losses.

Additional post-observation diagnostics assess sensitivity without changing the primary dataset: leave-one-trial-out means for publication rates and within-trial p99 values; leave-one-common-block-out ratios for all thirteen paired effects; first-five versus last-five block summaries; and summaries between recorded execution interruptions. Leave-one-out ranges measure influence, not confidence coverage. Early/late contrasts and rank correlations with block order are descriptive, with no trend significance test or causal interpretation. The interruption segments follow planned trial ranges D001–D021, D022–D030, D031–D037, D038–D042, and D043–D060, using completed primary observations only. Their unequal, small cell counts prevent treating these segments as independent controlled replications. All computations and membership lists are retained.

4.7 Protocol development and execution incidents

The local protocol was written before main collection but was not externally preregistered. Twelve initial pilot trials were excluded. Before main collection, we added receive-request timestamps, increased the backlog from 4096 to 16384 messages after the pilot revealed short high-concurrency runs, and added the five-second primary durability series in place of relying on prefill timings. A final twelve-trial pilot verified the revised harness. Both pilot datasets and the prospective protocol versions are retained. The execution log records these changes; they are not presented as an immutable external registration.

During collection, a stream-creation request exceeded the five-second client deadline before a measured interval began. The server logged an 8.0186-second stream-creation handler. We retained that failed setup and the original logs, added checkpointed continuation, and retried the condition with unchanged settings and a fresh stream name. No completed measurement was replaced. After trial 88, a Redis-always warm-up returned through an error path and its cleanup also timed out, masking the primary operation error. Its original stack and log are retained.

Administrative create, inventory, and deletion timeout settings were subsequently increased to 30 s. Redis default publication and acknowledgment socket timeouts and NATS response waits remained 5 s; the blocking-read overrides described above were unchanged. The durability series also stopped after 21 completed trials when a NATS-always warm-up publication timed out; the server logged approximately 4.9925 s in deletion/read-loop processing for that warm-up stream. No timed sample for its next condition had begun. We preserved the original log and all completed samples, then added append-only continuation to that series without changing its measurement path or deadlines. A further NATS-always warm-up publication timed out after 30 completed durability trials, with a corresponding 3.5595 s deletion/read-loop warning; the existing checkpointed binary then continued without a further code change. The causes of these interruptions are not established. Setup and warm-up availability are therefore reported separately from timed outcomes.

Planned timed-publication trial D038 (Redis-always) subsequently aborted with a socket read timeout. Its partial client timings and confirmed-request count were lost on process abort and cannot be reconstructed from the retained evidence. A later retained-state snapshot contained 1568 distinct application IDs and zero pending consumer acknowledgments; this cannot identify which publications the client had confirmed. We preserve the failure as the planned outcome and exclude its unavailable throughput and latency from completed-run estimates. An orchestration error during failure-ledger copying caused an unintended completed replay of D038; that replay is retained but excluded from primary statistics. Stopping that continuation also caused an operator interruption of D043, whose partial broker state was preserved and whose planned condition was rerun after cleanup. These operational deviations and the exact evidence are recorded in the artifact. No observed slow completed planned trial was removed. A plot of publication throughput by block was added as a descriptive diagnostic after observing this variability; it is not a preregistered hypothesis test.

Table 2 consolidates the 550 planned outcomes and six additional documented attempts. Its accompanying record-level inventory links each entry to retained evidence and leaves unavailable times and confirmation counts missing. Successful warm-ups are part of their completed attempts, not additional rows. The two excluded twelve-trial pilots and infrastructure provisioning are outside this inventory. This is retrospective accounting of known execution outcomes, not a prospective exclusion rule or a denominator for estimating setup availability or total workload-completion cost. In particular, the later successful D043 attempt cannot reconstruct the interrupted attempt’s client history.

Retained outcome	Count	Treatment in this study
Completed timing trials	419	300 drain, 59 publication, 60 serial
Active consumer recovery	120	Recovered identifiers and cleared pending state
No-reclaimer controls	10	Pending at reconciliation; nominal budgets only
Planned publication failure	1	D038; client timings unavailable
Additional setup failure	1	Tmain052; excluded from timed estimates
Additional warm-up failures	3	Tmain089, D022, D031; timed trials not begun
Operator interruption	1	D043; interrupted client trace unavailable
Completed diagnostic replay	1	D038; retained and excluded from estimates

Table 2: Retained campaign outcomes and additional documented attempts. The first four rows account for all 550 planned outcomes. The remaining six attempts are separate from those outcomes. Counts do not reconstruct missing durations or establish end-to-end availability.

5 Results

5.1 Recorded outcomes and evidence quality

All 300 planned backlog-drain trials completed and their 4,915,200 message records passed complete identifier-set, uniqueness, retained-count, and pending-state checks. The 60 serial sensitivity trials similarly reconciled 61,440 messages. The primary publication series has 59 completed planned trials with 12,434,648 confirmed appends and one retained Redis-always timeout. Its accidental completed replay is excluded as specified in Section 4. Publication-only integrity checks establish the retained count, not a complete payload readback. The analyzer reconstructs each reported throughput and percentile from raw timings and rejects inconsistent summaries or a missing failure ledger.

All 120 active recovery episodes returned the original identifier, acknowledged it, and left one retained message with zero pending acknowledgments; JetStream reported two deliveries in each episode. All ten Redis no-reclaimer controls remained pending at the end of their observation windows. Every victim exit was recorded as killed by a signal. The largest kill-invocation-to-observed-exit bracket was 2.291 ms. The final seven-pod snapshot reported zero container restarts and zero cgroup OOM kills. Recorded driver CPU-counter differences showed no throttling within completed timed trials; this does not establish an idle testbed or identify the cause of the recorded timeouts.

5.2 RQ1: failure age, reclamation, and observed redelivery

Table 3 and Figure 1 report shorter failure-to-redelivery intervals after later kills at the same configured timeout. At $\Delta = 250$ ms, the nominal remaining intervals are 225 and 62.5 ms. JetStream’s corresponding medians are 226.5 and 63.6 ms; Redis with a 10 ms reclaim sleep yields 230.2 and 67.9 ms. At $\Delta = 1000$ ms, the medians decrease from 907.1 to 257.9 ms for JetStream and from 903.5 to 255.4 ms for Redis with the shorter sleep. This direction is consistent with the residual-time decomposition in Theorem 3.1: later failure leaves less of the existing eligibility interval to wait out. Eligibility depends on message age and acknowledgment state; it can arise

Δ	α	System / sleep	$\Delta(1 - \alpha)$ (ms)	Median (ms)	Range (ms)
250	0.1	JetStream	225.0	226.5	[225.9, 226.9]
250	0.1	Redis / 10 ms	225.0	230.2	[228.4, 234.3]
250	0.1	Redis / 100 ms	225.0	265.4	[234.0, 313.9]
250	0.75	JetStream	62.5	63.6	[63.1, 64.5]
250	0.75	Redis / 10 ms	62.5	67.9	[63.9, 70.6]
250	0.75	Redis / 100 ms	62.5	134.0	[75.8, 161.2]
1000	0.1	JetStream	900.0	907.1	[905.4, 910.2]
1000	0.1	Redis / 10 ms	900.0	903.5	[900.4, 913.7]
1000	0.1	Redis / 100 ms	900.0	943.5	[911.7, 981.1]
1000	0.75	JetStream	250.0	257.9	[255.5, 260.6]
1000	0.75	Redis / 10 ms	250.0	255.4	[251.2, 269.4]
1000	0.75	Redis / 100 ms	250.0	319.4	[258.9, 339.4]

Table 3: Failure-to-redelivery latency for all recovery configurations. Each row contains ten killed-consumer episodes. $\Delta(1 - \alpha)$ is a nominal reference based on the scheduled age after client receipt, not a verified server-side lower bound. Ranges contain the minimum and maximum observed episode.

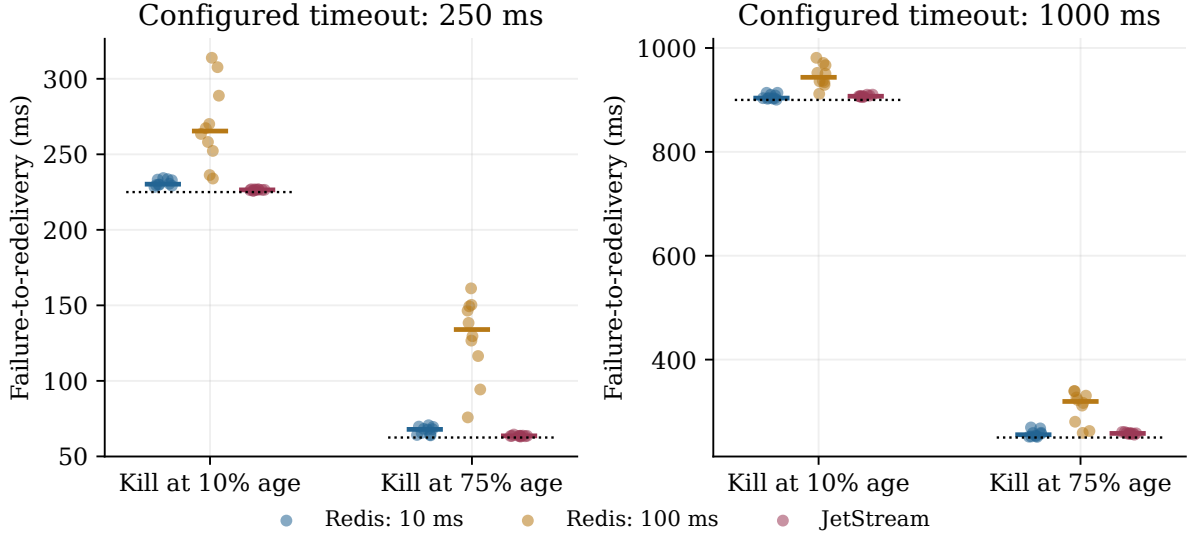


Figure 1: Every active-reclamation recovery observation. Horizontal colored marks show medians; horizontal jitter separates points and has no numerical meaning. Dotted lines show the nominal client-receipt-based residual-time reference. The ten no-reclaimer controls are censored and are reported separately rather than plotted as exact recovery times.

while a consumer remains alive. These observations do not establish a death-detection mechanism or directly observe the broker-side timer origin.

Redis reclamation frequency also matters. With a 100 ms sleep, the late-failure median rises to 134.0 ms at the 250 ms timeout and 319.4 ms at the 1000 ms timeout. Across the four age/timeout cells, the median observed excess $(\hat{r} - \hat{d}) - \Delta$ ranges from 4.0 to 6.2 ms with a 10 ms sleep, compared with 40.7–72.2 ms with a 100 ms sleep. JetStream’s median excess ranges from 1.9 to 8.6 ms. Neither a configured sleep nor an observed maximum is a proven future scheduling bound. JetStream’s 1000 ms medians are slightly above the shorter-sleep Redis medians, so the results do not support an unconditional claim that one system recovers faster.

The negative controls isolate a separate mechanism: ten observed non-recoveries with pending state intact are consistent with Proposition 3.3. Their finite observation procedures do not empirically prove infinite non-delivery; the proposition supplies that conditional counterexample from the stated new-read-only policy. Because exact censor endpoints were not recorded, these controls contribute descriptive evidence rather than exposure-time or survival estimates. The answer to RQ1 is therefore twofold: a failure leaves a residual timeout, and actual recovery additionally depends on reclamation or expiry selection and available demand. Under the active policies studied, all episodes recovered; without Redis reclamation, none of the controls did during observation.

5.3 RQ2: concurrency gains and cycle-latency cost

System	Mode	X_1 (kmsg/s)	X_{16} (kmsg/s)	X_{16}/X_1
Redis	memory	2.89 [2.74, 3.01]	36.37 [35.94, 36.83]	12.59 [12.13, 13.15]
Redis	periodic	2.66 [2.56, 2.76]	29.32 [27.73, 30.50]	11.02 [10.22, 11.79]
Redis	always	0.29 [0.29, 0.30]	2.51 [2.47, 2.55]	8.59 [8.47, 8.72]
JetStream	memory	2.30 [2.26, 2.34]	29.79 [29.43, 30.12]	12.94 [12.69, 13.23]
JetStream	periodic	2.27 [2.24, 2.29]	28.03 [26.89, 28.97]	12.36 [11.79, 12.83]
JetStream	always	2.24 [2.21, 2.27]	27.43 [27.04, 27.83]	12.24 [11.98, 12.54]

Table 4: Acknowledged backlog-drain throughput at one and sixteen consumers, with means and nominal 95% bootstrap intervals. Speedup is the ratio of condition means with a paired block-bootstrap interval. Each condition contains ten trials.

Increasing C from one to sixteen improved mean drain throughput in every profile (Table 4). Redis gains were 12.59, 11.02, and 8.59 times for memory, periodic, and always modes; JetStream gains were 12.94, 12.36, and 12.24 times. All six endpoint gains are below the sixteenfold increase in workers. The memory profiles reached 36.37 kmsg/s for Redis and 29.79 kmsg/s for JetStream at sixteen consumers. Periodic profiles reached 29.32 and 28.03 kmsg/s. These rates describe the measured request/acknowledgment path with a prefilled backlog, not a server-capacity ceiling or an open-loop saturation point.

The throughput gain has a cycle-latency cost. From one to sixteen consumers, mean within-trial cycle p99 increased from 0.543 to 0.867 ms for Redis memory and from 0.653 to 1.310 ms for JetStream memory. Redis periodic increased from 0.565 to 1.334 ms and JetStream periodic from 0.636 to 1.520 ms. Redis always increased from 5.942 to 10.311 ms, while JetStream always increased from 0.636 to 1.541 ms. In contrast, the corresponding common-start-to-receipt drain p99 values decrease between these endpoints (Appendix A): more workers complete the backlog sooner despite a slower individual cycle.

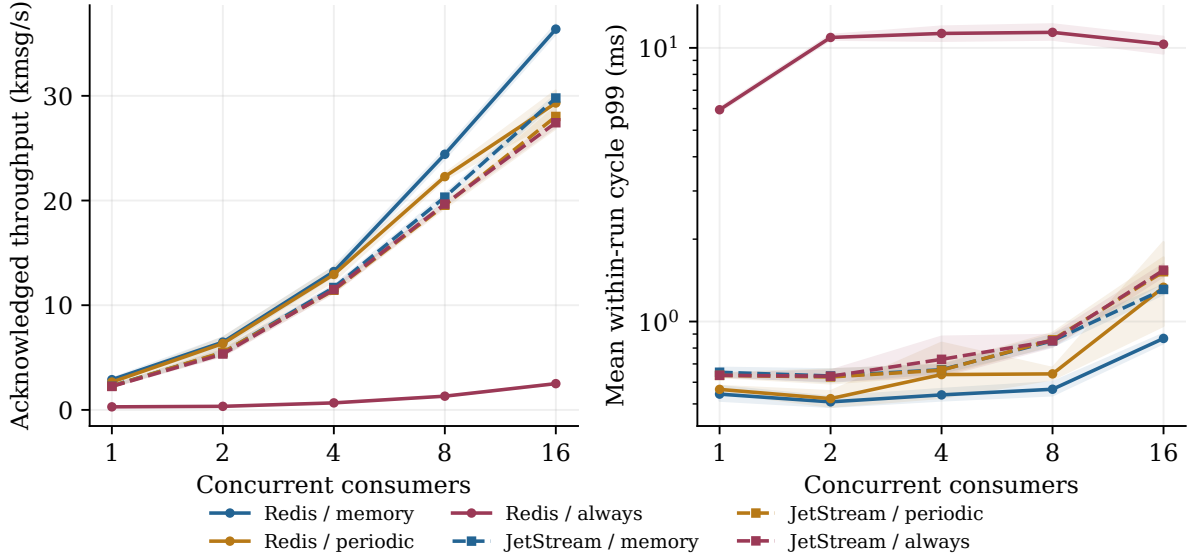


Figure 2: The complete concurrency response. Lines connect measured consumer counts without estimating unmeasured counts. Shaded bands are nominal 95% bootstrap confidence intervals for condition means; the right panel uses a logarithmic latency axis. Cycle latency is successful receive-request invocation to acknowledgment completion.

Reconstructed occupancy U lies between 0.9936 and 0.9995 across all 300 trials. Every worker’s recorded intervals are nonoverlapping and inside the measured window, satisfying the checked premises of Theorem 3.5. Thus recorded successful cycles occupy nearly all available worker time. The identity $X = CU/\bar{R}$ explains how increased mean cycle cost offsets the added concurrency; it does not identify which internal contention mechanism caused that cost. Nor does a p99 value replace $\bar{R}$ in this identity.

The always-mode endpoint is particularly easy to misread: JetStream drains 27.43 kmsg/s and Redis 2.51 kmsg/s, but their consumer-state synchronization contracts differ. RQ2 is answered by sublinear endpoint scaling with rising cycle tails within each profile. A cross-system ranking under an equal synchronous consumer-progress durability guarantee is not established by these configurations.

5.4 RQ3: confirmed-publication cost under persistence policies

The periodic configurations exhibited lower confirmed-publication throughput than the memory configurations in this campaign. Memory and periodic means were 73.90 and 61.38 kmsg/s for Redis, and 60.38 and 49.49 kmsg/s for JetStream. The paired periodic/memory throughput ratios were 0.8305 [0.8237, 0.8371] for Redis and 0.8197 [0.7933, 0.8511] for JetStream, using nominal 95% block-bootstrap intervals. Mean within-trial publication p99 was 0.474 versus 0.557 ms and 0.704 versus 0.849 ms, respectively. These descriptive differences include the evolving storage history and shared environment; they do not isolate the causal cost of synchronization policy or measure power-loss data loss.

Always-mode publication was substantially slower and more variable. The nine completed planned Redis-always trials averaged 3028 msg/s [1725, 4306], with a mean within-trial p99 of

System	Mode	n	Throughput (kmsg/s)	p50 (ms)	p99 (ms)
Redis	memory	10	73.90 [73.60, 74.19]	0.204	0.474 [0.466, 0.486]
Redis	periodic	10	61.38 [61.08, 61.67]	0.247	0.557 [0.551, 0.565]
Redis	always	9	3.03 [1.73, 4.31]	9.959	29.001 [11.615, 50.929]
JetStream	memory	10	60.38 [58.99, 61.32]	0.241	0.704 [0.678, 0.742]
JetStream	periodic	10	49.49 [48.29, 50.48]	0.281	0.849 [0.832, 0.869]
JetStream	always	10	0.37 [0.19, 0.56]	108.664	579.568 [91.362, 1,459.647]

Table 5: Five-second publication workload with sixteen independent publishers. Throughput and p99 entries give means and nominal 95% bootstrap intervals; p50 entries are means of within-trial medians. n is the number of completed planned trials out of ten; the failed Redis-always trial is reported separately.

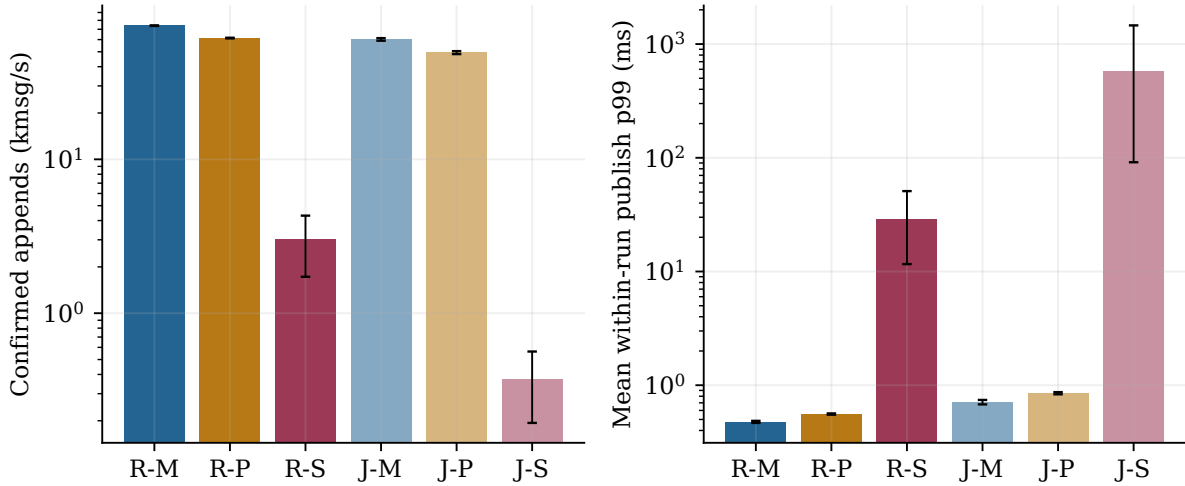


Figure 3: Confirmed-publication throughput and latency for Redis (R) and JetStream (J), in memory (M), periodic (P), and always (S) profiles. Both vertical axes are logarithmic. Error bars give nominal 95% bootstrap confidence intervals for means conditional on completion (nine Redis-always trials and ten for each other profile).

29.0 ms [11.6, 50.9]. The tenth planned trial timed out at the measured request deadline and has no reconstructible client latency sample. The ten completed JetStream-always trials averaged 375 msg/s [194, 564], with a mean within-trial p99 of 579.6 ms [91.4, 1459.6]. The wide intervals and per-block observations in Figure 4 are material results: the point estimates should not be presented as stable production tail-latency guarantees. The separately recorded setup and warm-up incidents further constrain operational interpretation.

System	Mode	Mean p99 (ms)	Median p99 (ms)	LOO means (ms)	Early / late rate (kmsg/s)
Redis	memory	0.47	0.47	[0.47, 0.48]	73.98 / 73.83
Redis	periodic	0.56	0.56	[0.56, 0.56]	61.38 / 61.37
Redis	always	29.00	7.46	[20.15, 32.10]	3.78 / 2.08
JetStream	memory	0.70	0.68	[0.69, 0.71]	59.68 / 61.07
JetStream	periodic	0.85	0.84	[0.84, 0.85]	50.35 / 48.64
JetStream	always	579.57	160.26	[150.53, 639.37]	0.45 / 0.30

Table 6: Post-observation publication sensitivity. The mean and median summarize completed trials’ p99 values. LOO gives the range of means after omitting each completed trial in turn, solely as an influence diagnostic. Early and late rates use blocks 1–5 and 6–10. Redis-always has five early and four late completions; other cells have five in each half. No observation is removed from primary estimates.

Table 6 shows the sensitivity of always-mode summaries. JetStream-always’s mean within-trial p99 is 579.6 ms, whereas the median of those ten p99 values is 160.3 ms. Its leave-one-out means range from 150.5 to 639.4 ms; the minimum omits D040, whose within-trial p99 is 4440.9 ms. Redis-always’s corresponding mean and median are 29.0 and 7.5 ms. These contrasts characterize influence and skew, not grounds for discarding slow observations.

System	Mode	Trials	Messages per trial	Positions above p99
Redis	memory	10	365,505–372,128	3,655–3,721
Redis	periodic	10	303,669–310,204	3,036–3,102
Redis	always	9	2,608–26,221	26–262
JetStream	memory	10	273,779–310,872	2,737–3,108
JetStream	periodic	10	243,055–257,750	2,430–2,577
JetStream	always	10	149–3,577	1–35

Table 7: Sample information underlying completed publication trials’ empirical p99 values. Ranges span the retained trials of each profile. Positions above p99 are $N - \lceil 0.99N \rceil$ and do not imply independent tail observations.

Table 7 makes the unequal sample sizes explicit. All 59 trial counts, ranks, and p99 values are retained in `data/derived/publication-tail-samples.csv`. D040 contains 149 confirmed publications, so its p99 is order statistic 148, the second-largest observation. Its influential tail estimate consequently has very little information above the selected rank. The mean of trial p99 values remains a descriptive summary of these observed trials; it is weak evidence for a stable population tail, and equal trial weights do not equalize percentile precision. Longer, independently repeated observations would be required for a stronger tail characterization. We retain D040 and do not replace its percentile with a pooled-message estimate.

The direction of all thirteen reported paired contrasts survives every leave-one-common-block-out calculation. Magnitudes remain conditional on the retained blocks: the periodic/memory publication ratios range from 0.8285 to 0.8320 for Redis and from 0.8076 to 0.8282 for JetStream in that diagnostic. Early versus late always-mode throughput declines from 3.78 to 2.08 kmsg/s for Redis and from 0.45 to 0.30 kmsg/s for JetStream. A simple monotonic degradation model is also insufficient: JetStream-always averages 483.3 msg/s in D001–D021, 24.6 msg/s in the interruption-bounded D038–D042 segment containing its single D040 observation, and 462.7 msg/s in D043–D060. The artifact reports every segment and cell count. These uncontrolled contrasts establish variability and sensitivity to execution history, not that an interruption or AOF growth caused a particular outcome. Independent campaign repetitions are needed to assess repeatability beyond this deployment.

Against memory mode, the paired always/memory throughput ratio is 0.0410 [0.0233, 0.0583] for Redis over nine common completed blocks and 0.00621 [0.00318, 0.00945] for JetStream over ten blocks. Their reciprocal point estimates are 24.4- and 161.1-fold slowdowns. Because the Redis estimate excludes an unavailable failed-trial metric, it describes successful runs and can understate the operational cost of the complete outcome distribution. We neither impute zero throughput nor replace the failed outcome with its diagnostic replay.

The serial-publisher sensitivity gives a different view (Appendix B): always-mode publication averaged 603 msg/s [591, 614] for Redis and 585 msg/s [565, 598] for JetStream. The close point estimates do not establish equivalence. This shorter fixed-count workload differs in both publication concurrency and duration, so it cannot isolate batching as the cause of the contrast with the sixteen-publisher results. No syscall or storage-device trace was collected to establish that mechanism.

RQ3 is therefore answered as a descriptive, workload-specific tradeoff: the configured periodic profiles preserve most memory-mode throughput with modestly higher observed tails, whereas always-mode confirmed publication incurs much larger and more variable observed costs in this shared Btrfs/NVMe environment. The synchronization contract, accumulated storage history, completion conditioning, and failure class must accompany that performance statement. Neither these append measurements nor the fast JetStream drain acknowledgments prove equivalent durable consumer progress or stable-media survival after a power cut.

6 Discussion

6.1 Recovery is a joint broker/client policy

The useful operational parameter is the residual timeout at failure, followed by the delay until the client obtains the eligible message. A consumer that fails late in its unacknowledged interval can have a shorter failure-to-redelivery time than one that fails early, even with identical settings. Comparing either interval directly with a full `AckWait` or minimum idle age would confound message age with recovery performance. Theorem 3.1 provides the appropriate decomposition, and Corollary 3.2 accounts for the unobserved initial network offset.

Redis makes reclamation scheduling an explicit application responsibility in the selected release. That permits an application-specific polling policy, while requiring correct cursor progress, protection against premature claims of slow work, and a balance between recovery latency and additional command traffic. JetStream supplies expiry tracking but still depends on retained consumer state and sufficient pull demand. A smaller timeout alone is not a free reliability improvement: if normal processing can exceed it, work may be redelivered while the original

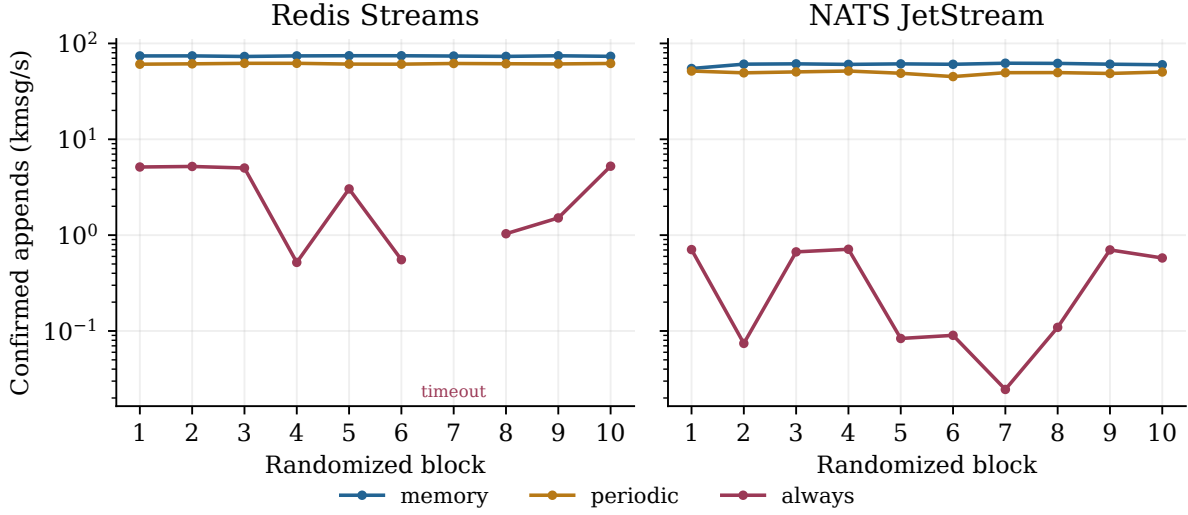


Figure 4: Descriptive publication-throughput diagnostic by randomized block. Each point is one completed planned trial. The timeout annotation has no throughput coordinate and identifies the failed Redis-always trial in block seven. A missing value interrupts its line; no failure is imputed as zero. Blocks are ordered experimental groups, not equal wall-time intervals.

consumer is still active. We do not measure long-running application effects or implement an application idempotency layer.

6.2 Concurrency gains and acknowledgment cost

The fixed-backlog workload asks how quickly this client path completes a bounded set of messages. More workers can overlap waiting on network responses or storage operations. At the same time, contention, wakeups, server serialization, and protocol traffic can increase per-message cycle latency. The exact occupancy identity distinguishes these two effects and prevents equating a throughput gain with a reduction in every latency metric. A queue’s drain-delay percentile may fall while its per-consumer cycle percentile rises.

The always profiles particularly require semantic care. Redis logs stream consumer-group mutations in its AOF path, while the pinned single-replica JetStream consumer uses an asynchronous state flusher. A fast JetStream acknowledgment path in that configuration is not evidence that an equivalent synchronous consumer-progress guarantee is cheaper. The primary append experiment measures publication confirmations separately, where the configured synchronization policy directly changes the intended append path. This separation is necessary for an interpretable answer to RQ3.

6.3 What the durability comparison supports

The configured profiles and repeated randomized trials support descriptive comparisons of confirmed-append cost in this environment. This is one component of the study’s comparison of specific recovery policies, concurrency behavior, and acknowledgment costs under documented conditions. Because persistent processes accumulate different storage histories and share the underlying device, the comparison cannot isolate the causal effect of synchronization policy or

identify the cause of individual stalls. It does not establish the stable-media contents after power failure, the loss probability under a disk error, or recovery time after a broker restart. A consumer kill leaves broker memory and the operating system page cache intact. Even killing the broker process would not by itself test host-power-loss behavior [26].

The synchronized profiles’ costs also depend on the filesystem, synchronization implementation, batching, and outstanding requests. We collect no syscall trace establishing a one-to-one relationship between messages and device flushes. The explanatory service bound in Section 3 is therefore conditional, and the serial-publisher sensitivity describes an additional measured workload rather than proving a particular internal batching mechanism.

7 Threats to validity

Construct validity. “Reliability” here comprises retained identities, observed completion counts, pending-state reconciliation, and consumer-failure redelivery. It does not mean exactly-once business effects, replicated consensus correctness, or power-failure durability. The six profiles align broad append-policy intentions but have different consumer-state and error-handling details. Client receipt is not broker delivery, and the failure instant is bracketed by a kill invocation and observed child exit. These are explicitly represented in the model and raw data.

Internal validity. The two-node homelab runs unrelated services and is neither CPU-pinned nor physically isolated. Randomization and repeated blocks reduce time-order confounding but cannot guarantee elimination of filesystem cache, background load, thermal, network, or control-plane effects. All six broker processes remain present, with only one treatment actively driven at a time; background synchronization can still outlive a measured interval. We record pod restarts, resource snapshots, cgroup CPU counters, effective configurations, startup logs, and image digests. Read-only resource snapshots run approximately every 30 s; monitoring overhead is not separately calibrated. AOF rewrite, eviction, stream trimming, and replication are intentionally excluded. In particular, disabling rewrite permits Redis AOF history to accumulate across logical deletions, while JetStream removes individual stream stores. This storage-history asymmetry is a specific carryover confound, not merely unspecified background noise. Final AOF sizes and the delayed-fsync counter establish its presence but not its performance effect. A fresh deployment campaign, or a justified common storage-reset protocol with its own warm-up treatment, is needed to investigate that effect.

Measurement validity. Both systems use the same Go driver and one TCP connection per logical worker, but different client libraries, protocols, and receive/acknowledgment round trips. Allocation, scheduler behavior, protocol parsing, and transport remain part of the measured path. The 256-message warm-up uses a different stream, so measured stream creation effects can persist after timing begins. Identifier and count reconciliation does not verify every byte of the stored payload. Five-second publication trials span several configured sync periods without establishing long-term stationarity. The one-publisher experiment has a different fixed message count and duration. Closed-loop request latency omits waiting that a fixed external arrival schedule might impose; no production p99 SLA is inferred.

Statistical conclusion validity. There are ten completed trials per concurrency and serial condition, ten planned publication trials per profile (nine Redis-always completions), and ten

recovery episodes per configuration; these are not millions of independent experiments. Publication intervals condition on completion and cannot correct selection bias from an unavailable failed-trial metric. Bootstrap intervals with nine or ten replications can be sensitive to skew, serial dependence, and outliers. They describe conditional uncertainty under the stated resampling model; coverage and stability across independent deployments remain unverified. The influence and execution-order diagnostics characterize fragility but do not establish block independence or establish nominal coverage. Pairing by randomized block improves comparability but leaves treatment times separated within a block. Recovery medians and ranges are descriptive at ten episodes per cell. We retain all observed trials and controls and distinguish their claims from the stronger hypothetical IID failure-probability calculation in Section 3.

External validity. The conclusions concern the stated releases, single-replica deployments, one Btrfs/NVMe storage environment with filesystem compression enabled, highly compressible 512-byte payloads, at most 16 consumers, and explicit synchronous acknowledgment APIs. New releases, larger or incompressible payloads, different filesystems or compression settings, batches, multiple streams, application work, large PELs, lossy networks, geographically separated deployments, quorum replication, storage-volume failover, or alternative clients can change the result. The study is not a substitute for workload-specific evaluation before a production choice. Recovery uses one pending message in the periodic profile without the concurrent publication workload; interactions between heavy load and failure recovery are not measured. The source audit is targeted and does not constitute a systematic review proving exhaustive novelty.

8 Artifact availability and reproducibility

The accompanying local artifact contains the complete LaTeX source, bibliography, generated vector figures, homelab manifests, a checked Lean model and verification receipts, Go harness with locked modules, prospective protocol versions, deviation log, raw compressed timing records, JSONL trial summaries, environment receipts, and deterministic analysis code. A checksum manifest identifies the delivered files. The bibliography pins server source revisions where mutable documentation could otherwise introduce version ambiguity. Pilot and main datasets are stored separately.

Reproducing the reported results begins with the retained campaign, not a new collection. The analysis verifies a frozen hash inventory of its raw data and environment records before recomputing identity checks, percentile values, throughputs, paired ratios, confidence intervals, occupancy bounds, and sensitivity diagnostics. Original records predate campaign identifiers; the hash inventory binds those exact bytes to the original adjudication ledger. Changed or additional campaign files are rejected. Tables and plot coordinates are generated from the verified values.

A separate collection workflow requires explicit campaign and attempt identifiers, a fresh namespace, and new output directories. Structured observations and failure records carry both identifiers; each collection is sealed with file hashes and checked for identity consistency. Collection cannot overwrite the reported campaign, and failed attempts are retained without automatic replay. The original measured harness is archived separately from the revised collection harness. The latter preserves partial publication observations after returned request errors or caught worker panics, records unconfirmed requests as unknown, and synchronizes their files before broker cleanup. Cleanup errors cannot replace the primary error. Abrupt loss of the driver before serialization can still destroy buffered observations. The revised recovery controller separately

records the nominal budget origin and deadline, receipt of a timer notification when applicable, completion of the survivor’s work and the controller’s wait, and the start and end of reconciliation. A receipt returned during shutdown is retained explicitly. These prospective safeguards cannot recover D038 or historical censoring endpoints and are not presented as having been used in the original measurements. Their functional validation is separate from the performance dataset. The README distinguishes exact data reanalysis, independent proof checking, and new experimental collection.

The accompanying artifact, version 1.1.0, has a release manifest (`release/manifest.json`), citation metadata (`CITATION.cff`), and a detached SHA-256 archive checksum. The original manuscript, research data, figures, and narrative documentation are licensed under Creative Commons Attribution 4.0 International; original software and formalization are licensed under MIT. `LICENSE.md` specifies the scope and third-party exceptions. The artifact map distinguishes historical measured sources, subsequent validated harness snapshots, and current analysis and document. Current validation receipts identify the reviewed PDF by its content hash; older receipts remain explicitly historical. Hashes establish consistency of the supplied bytes, not independent certification of collection time or completeness. A public archival identifier has not yet been assigned. The retained evidence permits local computational reproduction and independent examination; no externally executed replication or external artifact evaluation is claimed.

OpenAI Codex assisted with source investigation, experimental software, statistical analysis, formalization, manuscript drafting, and internal checking. Technical statements were checked against the cited primary sources, and numerical statements against retained homelab observations. Internal AI-assisted checking is distinct from independent scientific peer review. No generated or estimated benchmark result is substituted for a measured observation.

9 Conclusion

This study evaluated Redis Streams 7.4.2 and NATS JetStream 2.10.24 through explicit protocol analysis, conditional mathematical proofs, selected machine-checked model results, and retained homelab measurements. Its conclusions are bounded by single-replica deployments, the specified client paths, compressible 512-byte payloads, and one shared storage environment.

For RQ1, observed failure-to-redelivery intervals were consistent with the remaining eligibility timeout and subsequent recovery scheduling. Later kills were followed by shorter intervals in both systems, and longer Redis reclaim sleeps were associated with longer additional delays. All ten new-read-only controls remained pending. All 120 active recovery episodes completed with the original identifier and cleared pending state. The residual-time theorem explains this pattern under stated timing assumptions. Neither the observations nor the theorem establish death detection or guaranteed empirical deadlines.

For RQ2, mean acknowledged backlog-drain throughput with sixteen logical consumers was 8.59–12.94 times the one-consumer mean across profiles. Cycle p99 was higher while the backlog completed sooner. The verified finite-window occupancy conditions and the identity $X = CU/\bar{R}$ connect this sublinear scaling to the measured cycle cost. They do not establish an unbounded scaling law, and the different always-mode consumer-state persistence paths prevent interpreting that cross-system drain comparison as equal-guarantee performance.

For RQ3, periodic configurations had approximately 83% of Redis memory-mode publication throughput and 82% of JetStream memory-mode throughput. Always-mode publication exhibited

lower throughput and greater variability, including one failed planned Redis trial. Completed-run means and nominal intervals retain that conditioning; the failure and operational deviations remain visible. Leave-one-block-out diagnostics preserve the directions of paired contrasts, while influential observations, sparse tail information, execution-order variation, and unequal storage-history carryover limit precision and prevent isolating synchronization-policy effects. The separate serial sensitivity further shows why duration and request concurrency must accompany a throughput figure. No consumer-kill or publication-latency result establishes power-loss durability.

The contribution is an auditable, computationally reproducible comparison of specific recovery policies, concurrency behavior, and acknowledgment costs under documented conditions. The versioned, licensed artifact traces numerical claims to retained observations and formal claims to their premises. Future work should repeat the study across independent storage hosts and less-compressible payloads, hold duration fixed while varying publication concurrency, and add replication, large pending sets, open-loop load, and controlled broker/storage faults. These extensions would address external validity and failure classes that the present evidence leaves unresolved.

A Complete concurrency matrix

The following table reports all thirty conditions. Throughput entries are trial means with nominal 95% bootstrap intervals. Cycle p99 is the mean of within-trial request-to-acknowledgment p99 values; drain p99 is the mean of within-trial common-drain-start-to-receipt p99 values. Each cell has ten trials and a backlog of 16384 messages. These two latency definitions answer different questions.

System	Mode	C	Throughput (kmsg/s)	Cycle p99 (ms)	Drain p99 (ms)
Redis	memory	1	2.89 [2.74, 3.01]	0.543	5649.9
Redis	memory	2	6.47 [6.06, 6.90]	0.509	2532.3
Redis	memory	4	13.21 [12.82, 13.64]	0.539	1231.2
Redis	memory	8	24.42 [23.98, 24.83]	0.566	664.4
Redis	memory	16	36.37 [35.94, 36.83]	0.867	445.8
Redis	periodic	1	2.66 [2.56, 2.76]	0.565	6116.9
Redis	periodic	2	6.34 [5.81, 6.90]	0.523	2606.0
Redis	periodic	4	12.94 [12.31, 13.62]	0.640	1261.8
Redis	periodic	8	22.28 [22.02, 22.55]	0.644	728.0
Redis	periodic	16	29.32 [27.73, 30.50]	1.334	557.6
Redis	always	1	0.29 [0.29, 0.30]	5.942	55604.9
Redis	always	2	0.34 [0.33, 0.35]	10.915	47736.5
Redis	always	4	0.67 [0.66, 0.68]	11.291	24240.7
Redis	always	8	1.31 [1.30, 1.33]	11.395	12380.0
Redis	always	16	2.51 [2.47, 2.55]	10.311	6469.7
JetStream	memory	1	2.30 [2.26, 2.34]	0.653	7052.2
JetStream	memory	2	5.46 [5.25, 5.74]	0.633	2986.6
JetStream	memory	4	11.69 [11.52, 11.90]	0.667	1388.1
JetStream	memory	8	20.32 [20.05, 20.61]	0.851	798.4
JetStream	memory	16	29.79 [29.43, 30.12]	1.310	544.3
JetStream	periodic	1	2.27 [2.24, 2.29]	0.636	7155.0
JetStream	periodic	2	5.50 [5.20, 5.84]	0.627	2970.8
JetStream	periodic	4	11.43 [11.23, 11.65]	0.664	1421.1
JetStream	periodic	8	19.55 [19.28, 19.80]	0.857	829.6

System	Mode	C	Throughput (kmsg/s)	Cycle p99 (ms)	Drain p99 (ms)
JetStream	periodic	16	28.03 [26.89, 28.97]	1.520	580.6
JetStream	always	1	2.24 [2.21, 2.27]	0.636	7245.0
JetStream	always	2	5.34 [5.15, 5.61]	0.631	3051.9
JetStream	always	4	11.50 [11.20, 11.79]	0.728	1413.7
JetStream	always	8	19.66 [19.49, 19.81]	0.850	824.5
JetStream	always	16	27.43 [27.04, 27.83]	1.541	591.9

B Serial-publisher sensitivity

Each condition below uses one publisher with one request outstanding, 1024 messages per trial, and ten randomized blocks. The table is a separate fixed-count sensitivity experiment; it is not a change of publisher count under otherwise identical five-second trial duration.

System	Mode	Throughput (msg/s)	Publish p99 (ms)
Redis	memory	5,946.8 [5,385.6, 6,724.1]	0.240 [0.217, 0.257]
Redis	periodic	5,494.4 [5,076.8, 6,107.2]	0.257 [0.233, 0.278]
Redis	always	602.7 [591.3, 613.6]	2.847 [2.365, 3.340]
JetStream	memory	4,736.8 [4,479.8, 5,070.7]	0.325 [0.304, 0.351]
JetStream	periodic	4,911.5 [4,457.7, 5,425.8]	0.334 [0.304, 0.362]
JetStream	always	584.8 [564.8, 598.0]	2.756 [2.063, 3.807]

Table 9: Serial publication: means and nominal 95% bootstrap intervals.

C Formal assumptions and claim-to-evidence map

Claim	Evidence	Boundary
Residual recovery delay	Theorem 3.1; all fault timestamps	Requires retained state and explicit finite selection/demand-delay bounds
No new-read reclamation	Proposition 3.3; ten negative controls; pinned Redis source	Specific to the stated new-message-read-only policy and Redis release
Concurrency cost	Theorem 3.5; 300 trial traces	Finite workload and measured client path; no universal monotonic scaling claim
Persistence cost	60 planned publication outcomes, including one failure; 60 serial trials; effective configuration receipts	Confirmation and configured policy, without power-loss or I/O-fault validation
Stable-storage implication	Proposition 3.6	Abstract successful-and-honored synchronization contract; no implementation verification
Application effect ambiguity	Proposition 3.7	Separate non-idempotent effect and acknowledgment; not an impossibility result for transactional applications
Observed identities	Raw drained-message traces, complete ID-set checks, retained/pending counts	Publication-only trials reconcile counts without complete payload readback

References

- [1] Andrew D. Birrell and Bruce Jay Nelson. Implementing remote procedure calls. *ACM Transactions on Computer Systems*, 2(1):39–59, 1984. URL: <https://www.cs.cornell.edu/courses/cs614/2004sp/papers/BN84.pdf>, doi:10.1145/2080.357392.
- [2] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi:10.1093/biomet/26.4.404.
- [3] Manuel Coenen, Christoph Wagner, Alexander Echler, and Sebastian Frischbier. Benchmarking financial data feed systems. In *Proceedings of the 13th ACM International Conference on Distributed and Event-Based Systems*, DEBS ’19, pages 252–253. ACM, 2019. Poster; author manuscript deposited in 2020. URL: <https://arxiv.org/abs/2010.15534>, doi:10.1145/3328905.3332506.
- [4] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction: CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. URL: <https://lean-lang.org/papers/lean4.pdf>, doi:10.1007/978-3-030-79876-5_37.
- [5] HdrHistogram contributors. A high dynamic range (HDR) histogram. Software and coordinated-omission documentation. Accessed 2026-09-23. URL: <https://github.com/HdrHistogram/HdrHistogram>.
- [6] Tomas Kalibera and Richard Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the 2013 International Symposium on Memory Management*, ISMM ’13, pages 63–74. ACM, 2013. URL: <https://kar.kent.ac.uk/33611/>, doi:10.1145/2464157.2464160.
- [7] Kyle Kingsbury. NATS 2.12.1. Jepsen technical report, December 2025. Published 2025-12-08. Accessed 2026-09-23. URL: <https://jepsen.io/analyses/nats-2.12.1>.
- [8] Jaeyeon Lim, Sewan Gu, and Dongweon Yoon. Performance analysis of event-driven autoscaling in Kubernetes environment. *Journal of Korean Institute of Information Technology*, 24(8):159–169, 2026. Full publisher HTML inspected; article in Korean. Accessed 2026-09-24. URL: https://ki-it.com/_common/do.php?a=full&aidx=50551&b=12&bidx=4529, doi:10.14801/jkiit.2026.24.8.159.
- [9] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS XIV, pages 265–276. ACM, 2009. URL: <https://research.ibm.com/publications/producing-wrong-data-without-doing-any-thing-obviously-wrong>, doi:10.1145/1508244.1508275.
- [10] Tapajit Chandra Paul, Pawissanutt Lertpongjukorn, Hai Duc Nguyen, and Mohsen Amini Salehi. Benchmarking message brokers for IoT edge computing: A comprehensive performance study, 2026. Preprint, version 1. URL:

- <https://arxiv.org/abs/2603.21600v1>, arXiv:2603.21600, doi:10.48550/arXiv.2603.21600.
- [11] Redis. Redis persistence. Accessed 2026-09-23. URL: https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/.
 - [12] Redis. XACK command documentation. Accessed 2026-09-23. URL: <https://redis.io/docs/latest/commands/xack/>.
 - [13] Redis. XAUTOCLAIM command documentation. Accessed 2026-09-23; version-specific statements cross-checked against Redis 7.4.2 source. URL: <https://redis.io/docs/latest/commands/xautoclaim/>.
 - [14] Redis. XREADGROUP command documentation. Accessed 2026-09-23; version-specific statements cross-checked against Redis 7.4.2 source. URL: <https://redis.io/docs/latest/commands/xreadgroup/>.
 - [15] Redis contributors. Redis 7.4.2 annotated configuration, 2025. Source code, release 7.4.2. Accessed 2026-09-23. URL: <https://github.com/redis/redis/blob/a0a6f23d997b024689ba157916837f493a593a34/redis.conf>.
 - [16] Redis contributors. Redis 7.4.2 append-only file implementation: src/aof.c, 2025. Source code, release 7.4.2. Accessed 2026-09-23. URL: <https://github.com/redis/redis/blob/a0a6f23d997b024689ba157916837f493a593a34/src/aof.c>.
 - [17] Redis contributors. Redis 7.4.2 release, 2025. Released 2025-01-06. Accessed 2026-09-23. URL: <https://github.com/redis/redis/releases/tag/7.4.2>.
 - [18] Redis contributors. Redis 7.4.2 stream implementation: src/t_stream.c, 2025. Source code, release 7.4.2. Accessed 2026-09-23. URL: https://github.com/redis/redis/blob/a0a6f23d997b024689ba157916837f493a593a34/src/t_stream.c.
 - [19] Redis Go client contributors. go-redis v9.7.0 command-specific socket timeout calculation. Pinned source code. Accessed 2026-09-24. URL: <https://github.com/redis/go-redis/blob/ed37c33a9037483ad2a6b1042e5eb6df89009a1c/redis.go#L470-L478>.
 - [20] Redis Go client contributors. go-redis v9.7.0 stream command implementation. Source code. Accessed 2026-09-23. URL: https://github.com/redis/go-redis/blob/ed37c33a9037483ad2a6b1042e5eb6df89009a1c/stream_commands.go.
 - [21] J. H. Saltzer, D. P. Reed, and D. D. Clark. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4):277–288, 1984. URL: <https://web.mit.edu/saltzer/www/publications/endtoend/endtoend.pdf>, doi:10.1145/357401.357402.
 - [22] Bianca Schroeder, Adam Wierman, and Mor Harchol-Balter. Open versus closed: A cautionary tale. In *3rd Symposium on Networked Systems Design and Implementation*, NSDI '06, pages 239–252. USENIX Association, 2006. URL: <https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale>.

- [23] Gil Tene. wrk2: A constant throughput, correct latency recording variant of wrk. Software and measurement-method documentation. Accessed 2026-09-23. URL: <https://github.com/giltene/wrk2>.
- [24] The Lean Developers. Lean 4.24.0 release notes, October 2025. Released 2025-10-14. Source tag: leanprover/lean4 v4.24.0. Accessed 2026-09-23. URL: <https://lean-lang.org/doc/reference/latest/releases/v4.24.0/>.
- [25] The NATS Authors. JetStream consumers. Official documentation, revision f115becf6563e3bbe16bb94cbf87bfceb84199c1. Accessed 2026-09-23. URL: <https://github.com/nats-io/nats.docs/blob/f115becf6563e3bbe16bb94cbf87bfceb84199c1/nats-concepts/jetstream/consumers.md>.
- [26] The NATS Authors. JetStream: Syncing data to disk. Official documentation, revision f115becf6563e3bbe16bb94cbf87bfceb84199c1. Accessed 2026-09-23. URL: <https://github.com/nats-io/nats.docs/blob/f115becf6563e3bbe16bb94cbf87bfceb84199c1/nats-concepts/jetstream/README.md>.
- [27] The NATS Authors. nats.go v1.39.1 JetStream fetch and acknowledgment implementation. Source code. Accessed 2026-09-23. URL: <https://github.com/nats-io/nats.go/blob/4ba1afe8709b048ae89888f0f94da7eb8497e7ec/js.go>.
- [28] The NATS Authors. NATS Server v2.10.24 configuration parser: server/opts.go, 2024. Source code, release v2.10.24. Accessed 2026-09-23. URL: <https://github.com/nats-io/nats-server/blob/1d6f7eaf0bd966dd02c95b8d5f624868f1a62544/server/opts.go>.
- [29] The NATS Authors. NATS Server v2.10.24 consumer implementation: server/consumer.go, 2024. Source code, release v2.10.24. Accessed 2026-09-23. URL: <https://github.com/nats-io/nats-server/blob/1d6f7eaf0bd966dd02c95b8d5f624868f1a62544/server/consumer.go>.
- [30] The NATS Authors. NATS Server v2.10.24 file store: server/filestore.go, 2024. Source code, release v2.10.24. Accessed 2026-09-23. URL: <https://github.com/nats-io/nats-server/blob/1d6f7eaf0bd966dd02c95b8d5f624868f1a62544/server/filestore.go>.
- [31] The NATS Authors. NATS Server v2.10.24 release, 2024. Released 2024-12-17. Accessed 2026-09-23. URL: <https://github.com/nats-io/nats-server/releases/tag/v2.10.24>.